

Coding & Solution

Project Development Phase • AI/ML and GenAI Track

Key implementation details of the solution:

- Validation: `TextInput` enforces age 13–100, weight > 0 and ≤ 500 , goal in {general wellness, muscle gain, weight loss, flexibility, cardio fitness}, intensity in {low, medium, high}, and user IDs made of letters, digits, dash and underscore.
- Plan generation: `/generate-workout` calls `generate_workout_gemini`, then the nutrition tip (a tip failure is logged and the plan is still saved), then saves the user.
- Feedback loop: `/submit-feedback` loads the user, builds on the updated plan if present (otherwise the original), calls `update_workout_plan` and stores the revision.
- Gemini client: lazy client creation, 60 s timeout, up to 3 attempts with exponential backoff for codes 429/500/502/503/504, and `GeminiError` for empty or blocked responses.
- Errors: custom handlers return a styled error page for validation errors, HTTP errors and Gemini failures (HTTP 502).
- Frontend: a small script shows a loading overlay while the AI is working and restores the page on back navigation.

Prompt design (workout plan)

The prompt sets the role ("FitBuddy, a responsible fitness planning assistant"), supplies the user's details, and lists requirements: Day 1–7, a 5–10 minute warm-up, main workout with exercises and sets/reps or duration, rest, cooldown, gradual progression, no extreme practices, and advice to consult a professional if there is pain, injury or illness. Plain-text output is requested so it displays cleanly.

Database model

Column	Type	Notes
<code>user_id</code>	<code>String(100)</code>	Unique, indexed
<code>username</code>	<code>String(100)</code>	Required
<code>age / weight</code>	<code>Integer / Float (kg)</code>	Required
<code>goal / intensity</code>	<code>String</code>	From fixed lists
<code>original_plan / updated_plan</code>	<code>Text</code>	First plan and latest revision
<code>nutrition_tip</code>	<code>Text</code>	Optional